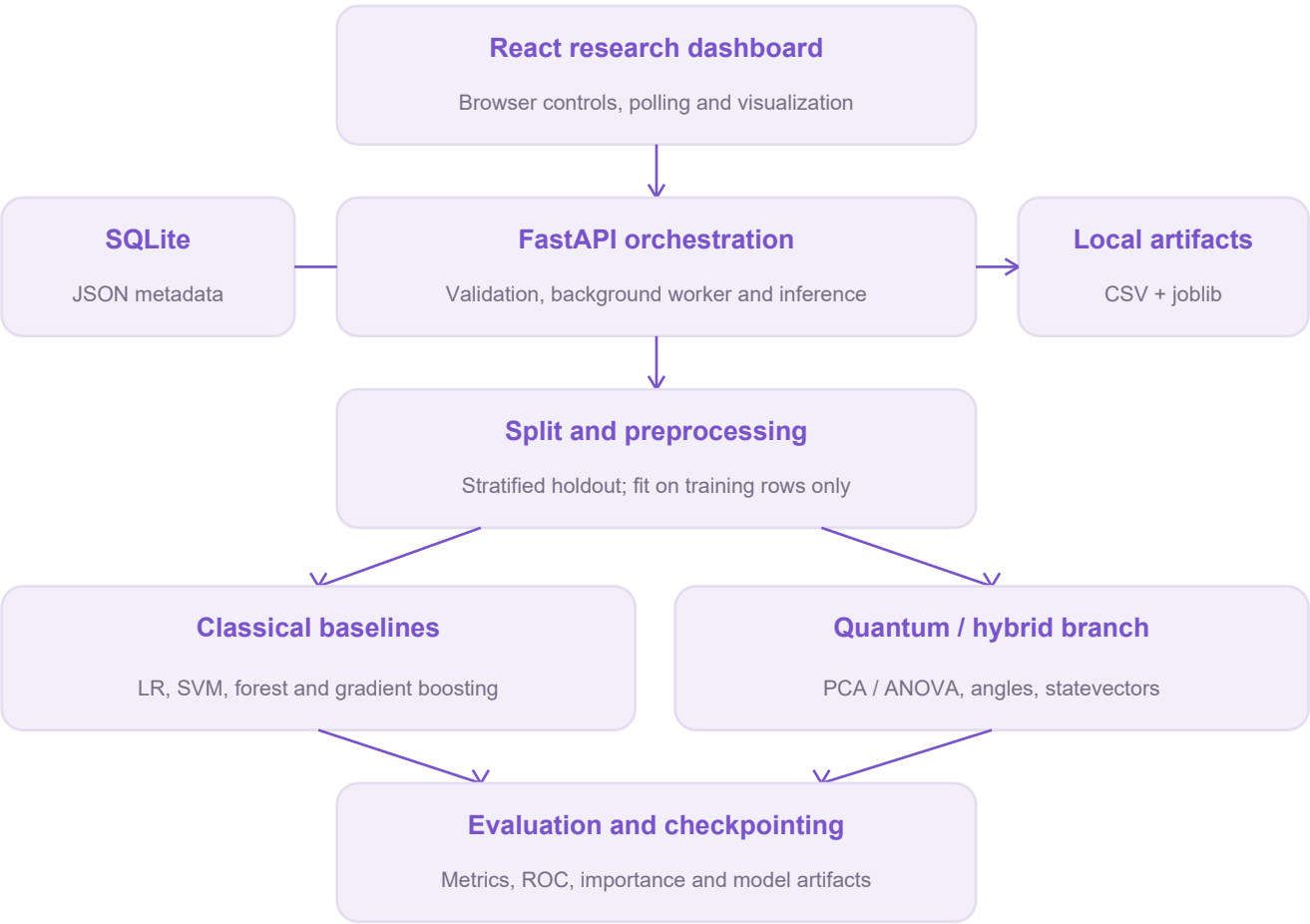


Hybrid Quantum-Classical Machine Learning Platform for Early Disease Detection

Architecture, implementation, experimental evidence and development roadmap



REPORT DATE	IMPLEMENTATION	EVIDENCE BASIS
1 October 2026	Research MVP v1.0	Source code + saved experiment

Prepared for the project owner. This report describes the delivered software as it exists in the workspace; proposed functionality is identified separately. No authorship, institution, clinical approval or quantum advantage is assumed.

01 / TECHNICAL PROJECT REPORT

Executive summary

Qura is an end-to-end local research application for exploring binary biomedical classification with both classical and simulated quantum approaches. It connects a React dashboard to a Python FastAPI service, a reproducible preprocessing and training pipeline, persistent experiment records, saved models and sample inference.

The original brief proposed a broad platform covering multiple diseases, advanced explainability, hardware execution and production privacy controls. The delivered system establishes the core workflow using the Wisconsin Breast Cancer benchmark. It is a working research MVP, not a completed clinical product or every deliverable in that brief.

What the platform delivers

- Dataset discovery, CSV upload, target-column selection, preview and quality summaries.
- Training-only preprocessing, stratified holdout evaluation, PCA or ANOVA reduction and reproducible seeds.
- Four classical classifiers, a statevector quantum-kernel SVM and a variational quantum classifier.
- Background training, SQLite experiment tracking, joblib checkpoints, measured metrics, ROC curves, confusion matrices and permutation importance.
- New-sample inference, probability estimates and JSON report exports through a responsive dashboard.

What the evidence shows

The latest complete saved experiment used 143 held-out samples. SVM achieved the highest thresholded accuracy (99.30%); random forest achieved the highest ROC-AUC (0.9979). The quantum kernel achieved 93.01% accuracy and ROC-AUC 0.9950. The VQC achieved 65.03% accuracy and ROC-AUC 0.5449 under a deliberately small optimization budget.

Interpretation

These are single-split demonstration results. Classical models trained on 426 rows while quantum models used 160. The results establish software functionality; they do not establish quantum advantage, screening effectiveness or clinical safety.

02 / TECHNICAL PROJECT REPORT

Report guide and evidence provenance

SECTION	TOPIC
01	Executive summary
02	Report guide and evidence provenance
03	Problem, objectives and intended use
04	Scope and delivery status
05	System architecture and technology stack
06	Dataset management and validation
07	Preprocessing and feature engineering
08	Classical baseline models
09	Quantum models and simulator
10	Quantum circuit analysis and limitations
11	Frontend product experience
12	Backend API and persistence
13	Training and experiment lifecycle
14	Evaluation methodology
15	Measured benchmark results
16	Results interpretation and explainability
17	Prediction workflow
18	Testing and quality assurance
19	Security, privacy and deployment
20	Installation and operating guide
21	Limitations, roadmap and conclusion
22	Appendix: configuration and source map
23	Appendix: references and glossary

Primary project evidence: backend/main.py, backend/quantum.py, backend/test_platform.py, src/ResearchApp.tsx, src/research.css, package.json, Dockerfile and README.md. Experimental numbers are read directly from backend/storage/research.db; no results are invented or inferred from interface artwork.

The report preserves the most recent complete six-model experiment as an evidence snapshot. The date on the report follows the user workspace date, 1 October 2026. Experiment timestamps are stored in UTC.

03 / TECHNICAL PROJECT REPORT

Problem, objectives and intended use

Research problem

Biomedical classification often combines noisy measurements, correlated predictors, missing values and limited sample sizes. A useful research platform must make data preparation, model comparison and interpretation transparent. Quantum models introduce additional constraints: feature dimension must fit the available qubits, circuit simulation can be costly, and an apparently strong benchmark score does not by itself demonstrate a computational advantage.

The project investigates whether a practical hybrid workflow can connect biomedical features to quantum-compatible representations and evaluate those models beside standard classical baselines. The current demonstration predicts benign versus malignant labels from derived cell-nucleus measurements; it does not evaluate whether a disease is detected earlier in a real clinical pathway.

Project objectives

- Create one repeatable workflow from validated data to saved models and new-sample inference.
- Fit preprocessing only on training data and preserve experimental configuration.
- Compare classification quality and local execution time across classical and hybrid approaches.
- Expose feature importance, circuit structure and model outputs in a usable interface.
- Report measured findings objectively, including poor-performing quantum models.

Intended users and use boundary

The application is intended for students, researchers and developers working with public or anonymized tabular data in a trusted local workspace. Outputs support experimentation and technical discussion. They are not patient-specific medical advice, a diagnosis, or a substitute for validated clinical procedures.

Definition of success

Success at this stage means that the complete software workflow is runnable, artifacts persist, metrics are derived from held-out predictions, and limitations are visible. Clinical usefulness would require independent validation and a substantially different governance and deployment process.

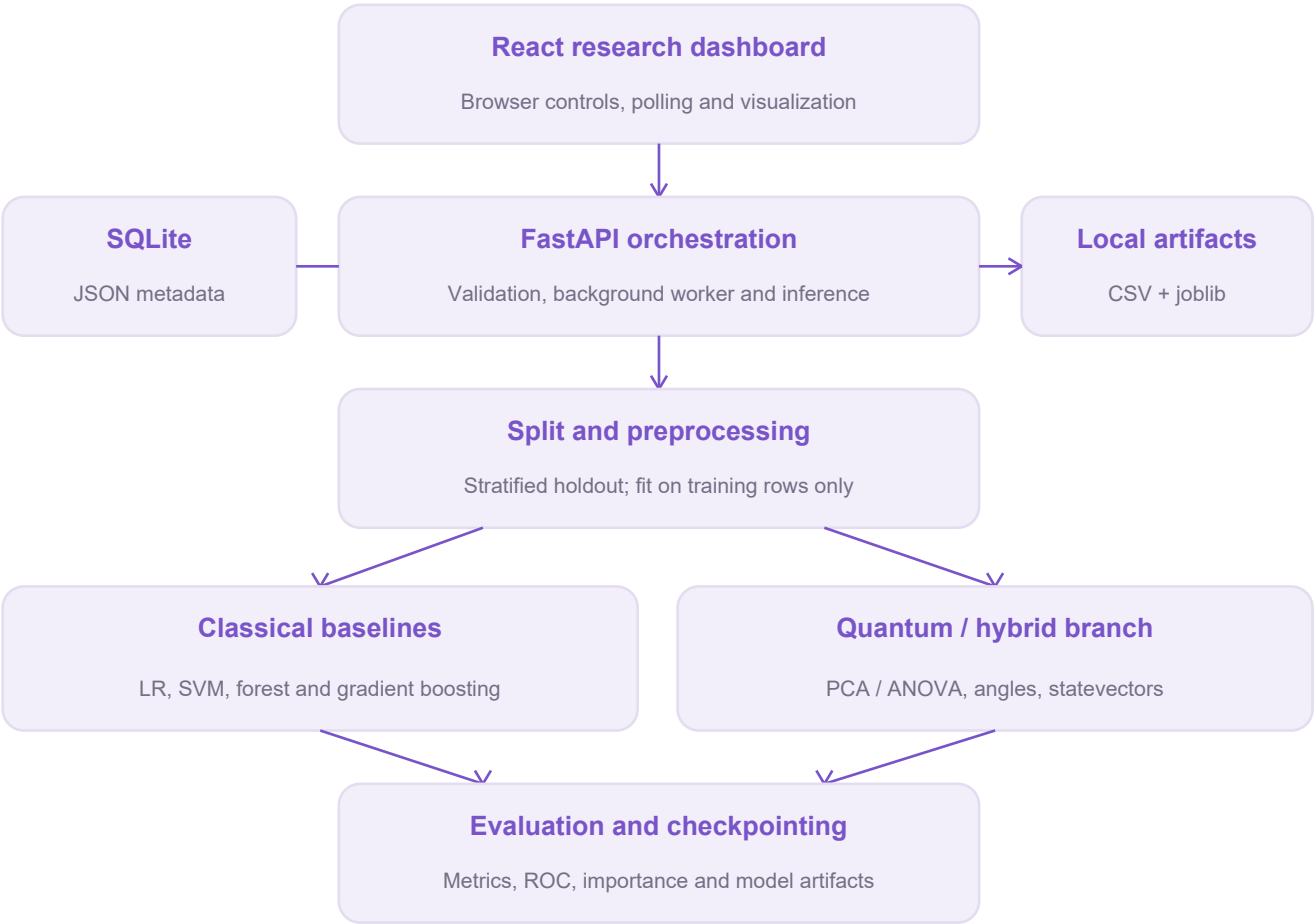
04 / TECHNICAL PROJECT REPORT

Scope and delivery status

MODULE / REQUEST	CURRENT STATUS
Dataset upload and preview	Implemented for binary-label CSV data; one bundled benchmark.
Preprocessing and features	Median/mode imputation, scaling, one-hot encoding, PCA or ANOVA; exact duplicates removed before splitting.
Classical models	Logistic regression, SVM, random forest and histogram gradient boosting.
Quantum models	Quantum kernel SVM and VQC implemented with a custom NumPy statevector simulator.
Evaluation and experiments	Holdout metrics, ROC, confusion matrices, timing, persistent configurations and checkpoints.
Explainability	Global permutation importance. SHAP and LIME are not implemented.
Predictions and exports	JSON samples, model probability output and JSON research exports.
Frontend and backend	Responsive dashboard and documented FastAPI API are implemented.
Multi-disease benchmarks	CSV uploads can represent other binary tasks; additional bundled diseases are not provided.
Hardware / Qiskit / PennyLane	Not configured; simulation uses the project implementation.
Cross-validation and calibration	Not implemented; results are single holdout estimates.
Production privacy and tenancy	No authentication or tenant isolation; trusted local use only.
Reports and presentation	This static PDF report is delivered separately; in-app PDF generation and a presentation deck remain absent.

The existing repository began as an AI campus-navigation interface. The active entry point now loads the Qura research application. The earlier App component was preserved as src/CampusApp.tsx with its legacy modules; retaining those files does not make them part of the new research workflow.

System architecture and technology stack



LAYER	IMPLEMENTATION	RESPONSIBILITY
UI	React, TypeScript, Vite	Navigation, configuration, forms, result views and polling.
API	FastAPI, Pydantic, Uvicorn	Validated requests and local service orchestration.
Data / ML	pandas, NumPy, scikit-learn	CSV handling, preprocessing, classifiers and metrics.
Quantum	NumPy + SciPy COBYLA	Exact states, kernel overlaps and VQC optimization.
Persistence	SQLite + CSV + joblib	Metadata, uploaded datasets and fitted pipelines.

During development the browser accesses Vite on port 3000, which proxies /api to FastAPI on port 5000. A built dist directory can also be served by FastAPI. The quantum estimator interface follows scikit-learn conventions so the same Pipeline can apply transforms before prediction.

06 / TECHNICAL PROJECT REPORT

Dataset management and validation

The bundled demonstration dataset is loaded with scikit-learn load_breast_cancer. It contains 569 samples and 30 numerical measurements. The project converts the library target into readable diagnosis labels. Dataset summaries expose sample count, columns, numerical features, missing values, duplicates, class distribution and six preview records. [R1]

BENCHMARK PROPERTY	VALUE
Dataset	Wisconsin Diagnostic Breast Cancer
Samples / numerical features	569 / 30
Benign / malignant	357 / 212
Target column	diagnosis
Negative / positive class	benign / malignant
Feature families	Radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension
Measurement summaries	Mean, standard error and worst-value summaries

CSV upload contract

POST /api/datasets/upload accepts a multipart file and target field. The filename must end in .csv, the read size is limited to 10 MB, and pandas must be able to parse the contents. The table must contain 30-10,000 rows, at most 200 columns, at least one predictor, and exactly two target classes with at least five rows each. Missing target values are rejected.

Accepted files receive server-generated UUID storage paths rather than user-controlled paths. Dataset metadata includes the original display filename and a small preview. The upload handler checks for duplicated column labels after parsing; pandas may rename duplicate headers during parsing, so this is not a complete original-header uniqueness guarantee.

Label semantics and constraints

Labels are sorted using their string representations and the final class is positive. For the benchmark this means malignant is positive. For another uploaded task the positive class must be checked before interpreting sensitivity or probability. Upload validation does not establish that data are anonymized, clinically meaningful, free of hidden leakage or valid after duplicate removal.

Preprocessing and feature engineering

Leakage-aware sequence

Training reads the CSV, removes exact duplicate rows, separates predictors from the target and performs a seeded stratified train/test split. The preprocessing pipeline is then fitted on the model training data. It is reused unchanged for held-out evaluation and inference. This follows the general training-only transformation principle documented by scikit-learn. [R2]

TRANSFORMATION	IMPLEMENTED BEHAVIOR
Numerical missing values	Median imputation; empty columns retained by SimpleImputer.
Numerical scaling	StandardScaler, learned from training rows.
Categorical missing values	Most-frequent imputation.
Categorical encoding	OneHotEncoder with unknown-category handling and dense output.
Duplicate handling	Remove exact full-row duplicates before splitting.
Quantum reduction	PCA or SelectKBest using ANOVA F statistics.
Quantum angle mapping	MinMaxScaler to $[-\pi, \pi]$, with clipping at inference.

Quantum-compatible dimensionality

The requested qubit count is bounded to 2-6. Effective components are limited by the requested count, the number of original input columns and the initial training sample count. The full preprocessing/reduction pipeline is fitted on the quantum training subset when the 160-row budget applies; it does not inherit transforms fitted on the larger classical training partition.

Current feature-engineering boundaries

There are no correlation heatmaps, nonlinear embeddings, outlier clipping or synthetic oversampling. Balanced class weights are used only by supported classical estimators. Highly cardinal categorical inputs can create a large dense matrix. Removing duplicates can leave too few examples for a valid stratified split; the job then fails and records a generic failure state.

Configuration preview

The `/api/preprocess` endpoint describes settings and checks schema/dataset existence. It does not materialize a processed dataset or prove that every chosen configuration will train successfully.

Classical baseline models

MODEL	ESTIMATOR / SETTINGS	ROLE
Logistic regression	LogisticRegression; max_iter=1000; seeded; optional balanced weights	Compact probabilistic baseline.
Support vector machine	SVC; probability=True; default RBF kernel; seeded; optional balanced weights	Nonlinear baseline with probability estimates.
Random forest	RandomForestClassifier; 120 trees; seeded; n_jobs=1; optional balanced weights	Ensemble baseline for nonlinear structure.
Gradient boosting	HistGradientBoostingClassifier; seeded; remaining estimator defaults	Histogram-based boosting alternative to XGBoost.

Shared evaluation, different algorithms

All classical models use the same original features and the same held-out test rows. Each owns its fitted preprocessing pipeline, which is saved alongside the estimator. The training controls expose model selection and common experiment settings; they do not expose a full per-estimator hyperparameter search interface.

The current implementation does not fit XGBoost itself. Histogram gradient boosting supplies the equivalent boosting baseline permitted by the brief. When `balance=True`, logistic regression, SVM and random forest receive `class_weight="balanced"`. The current gradient-boosting construction does not apply balancing.

Interpreting baseline quality

A high ROC-AUC measures the ability to rank positive examples above negative examples across thresholds. It need not identify the model with the best sensitivity at the fixed 0.5 threshold. Accuracy is also influenced by prevalence: the benchmark has more benign than malignant rows. The report therefore presents sensitivity, specificity and confusion counts beside AUC.

Avoid selection on the test set

The dashboard highlights the largest measured AUC for exploration. Repeatedly selecting architectures or hyperparameters using this same holdout can bias the final reported score. A future study should use training/validation folds for selection and an untouched final evaluation set.

Quantum models and simulator

Exact statevector execution

backend/quantum.py allocates a complex vector with 2^n amplitudes, initializes the all-zero state and applies real RY rotations to each wire. Each layer can add trainable RY rotations followed by a ring of controlled-X gates. A gate update operates directly on basis-state amplitudes. The simulator uses exact probabilities, without finite-shot sampling or a device noise model.

MODEL	TRAINING AND INFERENCE
Quantum kernel SVM	Encode each row into a state. Compute $K(i,j)= \langle \psi_i \psi_j \rangle ^2$. Fit a probabilistic SVC with kernel="precomputed". Compare inference states to saved training states.
Variational quantum classifier	Initialize trainable angles from a seeded normal distribution. Minimize binary cross-entropy using SciPy COBYLA. Positive probability equals summed probabilities of basis states where qubit 0 is one.

Encoding and parameterization

Reduced inputs are mapped to $[-\pi, \pi]$ before angle encoding. A depth-d VQC on n wires has $n \cdot d$ trainable rotation angles. Probabilities are clipped to $[0.000001, 0.999999]$ for stable logarithms. The saved estimator stores optimized weights or training states and its classical SVM.

Optimization budget and scale

The public iterations field is passed to COBYLA as maxiter, which is a maximum number of objective function evaluations rather than a guaranteed number of full optimization iterations. [R3] Current validation accepts 10-200 evaluations. Quantum training is restricted to 160 seeded stratified rows where the training partition is larger.

Implementation identity

These models use a custom exact NumPy simulator. They do not execute on Qiskit, PennyLane or a physical quantum device. Statevector evaluation implements circuit mathematics, but all computation here occurs on classical hardware.

10 / TECHNICAL PROJECT REPORT

Quantum circuit analysis and limitations

VQC circuit sequence

```
Input x -> RY(x) on every wire
Repeat depth times:
  RY(theta[layer, wire]) on every wire
  CX(wire -> next wire), including the ring closure
Measure P(qubit 0 = 1) -> binary class probability
```

The ring gates are applied sequentially in the simulator. The interface displays a conceptual configuration preview, not a compiled hardware schedule. The configured depth counts repeated layers; it is not a reported transpiled gate depth. Effective feature components can be smaller than the requested qubit count, especially for narrow uploaded datasets.

Important kernel property of the current circuit

For the quantum kernel, the entangling layers are the same input-independent unitary U for every encoded state. Therefore $|\langle U \psi(x) | U \psi(z) \rangle|^2$ equals $|\langle \psi(x) | \psi(z) \rangle|^2$ because $U^\dagger U$ is the identity. These final shared entanglers cannot change the exact overlap kernel. Changing kernel circuit depth alone does not increase its representational power in this implementation.

```
K(x, z) = |⟨ψ(x) | ψ(z)⟩|^2
= product_j cos^2((angle_x[j] - angle_z[j])/2)
```

The product expression follows from the independent RY feature encoding. It can be evaluated classically without constructing statevectors. This is an implementation-level mathematical observation, not an experimental claim. The current kernel is a useful integration baseline but cannot substantiate a uniquely quantum advantage.

Constraints for future quantum research

A richer study should introduce explicitly input-dependent entangling feature maps or data re-uploading, investigate trainability across multiple seeds, use matched training budgets and compare against classical kernels on the same reduced features. Hardware experiments would additionally require shot counts, calibration metadata, device connectivity, transpilation and noise-aware evaluation.

Frontend product experience

The Qura dashboard replaces the active campus UI with a purple-and-neutral research workspace. A persistent sidebar gives access to all stages; status messages report API availability, background training progress and request failures. Responsive styles collapse the sidebar to named icon controls on narrower screens.

PAGE	USER-FACING FUNCTION
Overview	Dataset counts, best measured AUC, class distribution, model bars and workflow shortcuts.
Datasets	Choose a dataset, upload CSV with target name, inspect quality counts and preview records.
Preprocessing	Set split, seed, quantum reduction, qubits, layers and VQC budget; validate settings.
Model training	Select one or more of six models and start a background experiment.
Quantum circuit	Preview angle encoding, trainable rotations and ring connections.
Evaluation	Compare metrics; select a row for ROC and confusion matrix; export JSON results.
Explainability	Select a model and display its largest permutation-importance values.
Predictions	Load a benchmark sample or edit JSON, select a model and obtain probability output.
Experiments	Inspect run identifiers, UTC timestamps, completion counts and failure states.
Documentation	Short workflow guide; references repository README.

The UI polls datasets, models and experiments every three seconds. The simulator-connected badge indicates API availability rather than a separate hardware connection. Performance bars show the latest saved result per model type for the active dataset. The displayed workspace profile is presentation text; it is not an authenticated account.

Frontend boundaries

Prediction input is JSON rather than batch CSV upload. The new UI has manual browser verification, but no dedicated automated component or browser test suite. Global explainability bars are not individual patient explanations.

12 / TECHNICAL PROJECT REPORT

Backend API and persistence

METHOD / ROUTE	CONTRACT
GET /api/health	Service readiness, simulator identity and research-only flag.
GET /api/datasets	List stored dataset summaries.
GET /api/datasets/{id}	Return one dataset summary or 404.
POST /api/datasets/upload	Multipart CSV and target; return validated dataset record.
POST /api/preprocess	Read configuration and describe the preprocessing strategy.
POST /api/models/train	Validate selection and queue one experiment.
GET /api/models	List saved model metrics and metadata.
GET /api/models/{id}/metrics	Return the stored model result.
GET /api/models/{id}/explain	Return global permutation importance and its method label.
POST /api/models/predict	Accept model_id and 1-100 sample dictionaries.
GET /api/experiments	List persisted experiment runs.
GET /api/experiments/{id}	Return progress, configuration and status.

Storage design

SQLite uses a records table with id TEXT PRIMARY KEY, kind TEXT and body TEXT. Kind separates dataset, model and experiment records; JSON stores their flexible metadata. CSV files are named by dataset ID, while joblib files are named by model ID. Parameterized SQL is used for metadata lookups and writes. This is a compact local design, without per-user ownership or relational foreign-key enforcement.

Failures and schema validation

Pydantic validates numeric bounds and sample-list length. The training handler rejects unknown model names and invalid selection methods. Requests report 404 for unknown records, 409 when a run is already marked active, 413 for oversized files, and 422 for upload/configuration/inference validation failures. FastAPI supplies interactive OpenAPI documentation at /docs.

13 / TECHNICAL PROJECT REPORT

Training and experiment lifecycle

STAGE	RECORDED / EXECUTED BEHAVIOR
Create	Generate experiment UUID; save running status, total model count, configuration and timestamp.
Queue	Submit work to a ThreadPoolExecutor with one worker.
Prepare	Load CSV; remove duplicate rows; produce a seeded stratified split.
Fit	Construct model pipeline, apply any quantum sample budget and fit transformations plus estimator.
Evaluate	Calculate held-out probabilities, thresholded labels, metrics, timing and feature importance.
Checkpoint	Save pipeline with joblib; save model result and increment completed count.
Finish	Mark experiment completed after all requested models finish.
Failure / restart	Record failed state with a generic error. On import, mark previously running records failed.

What a model record contains

Each model stores its own identifier, experiment and dataset links, model name, positive class, full requested configuration, training/test sample counts, metrics, confusion matrix, ROC points, feature importance, timing and UTC creation timestamp. The fitted pipeline provides everything needed to apply the same transformations at inference.

Reproducibility and operational limits

The shared seed controls splitting and seeded estimators. VQC initialization is deterministic under the selected seed. Repeated runs can reproduce predictions for the same software environment; timings still vary with machine load. Dependency requirements use version ranges rather than a locked Python environment, so library defaults may change after upgrades.

The running-status check is not a transactionally protected global lock. Multiple concurrent API requests or multiple server processes can undermine its intent. The worker is thread-based, not a durable job queue, and there is no cancellation or resume endpoint. Successful model artifacts can remain if a later model in the same experiment fails.

Evaluation methodology

Demonstration protocol

The reported experiment uses seed 42 and a 25% stratified holdout: 426 training rows and 143 test rows. The test set contains 90 benign and 53 malignant samples. Classical models use the full training partition. Quantum pipelines fit on a 160-row stratified subset, with two PCA components, one circuit layer and a VQC budget of ten objective evaluations. All models are evaluated on the same test rows.

METRIC	DEFINITION / READING
Accuracy	$(TP + TN) / \text{all test rows}$.
Precision	$TP / (TP + FP)$; purity of positive predictions.
Sensitivity / recall	$TP / (TP + FN)$; fraction of positives detected.
Specificity	$TN / (TN + FP)$; fraction of negatives correctly rejected.
F1	Harmonic mean of precision and sensitivity.
ROC-AUC	Area under the true-positive vs false-positive rate curve.
PR-AUC field	scikit-learn average precision, not trapezoidal PR integration.
Training time	Wall time for pipeline construction/fitting; excludes evaluation, importance and checkpoint writes.
Inference time	Batched probability-prediction wall time divided by test sample count.

Threshold and uncertainty

Classification uses positive probability ≥ 0.5 . The experiment does not tune that threshold or calibrate it for a clinical objective. No confidence intervals, cross-validation folds or independent external validation cohorts are recorded. Runtime figures are local measurements, not a controlled hardware benchmark.

Fairness of the comparison

Same-test-set evaluation is useful, but training volume and feature representation differ between branches. A future comparison should match sample counts and reduced features and repeat splits before drawing conclusions about algorithmic benefit.

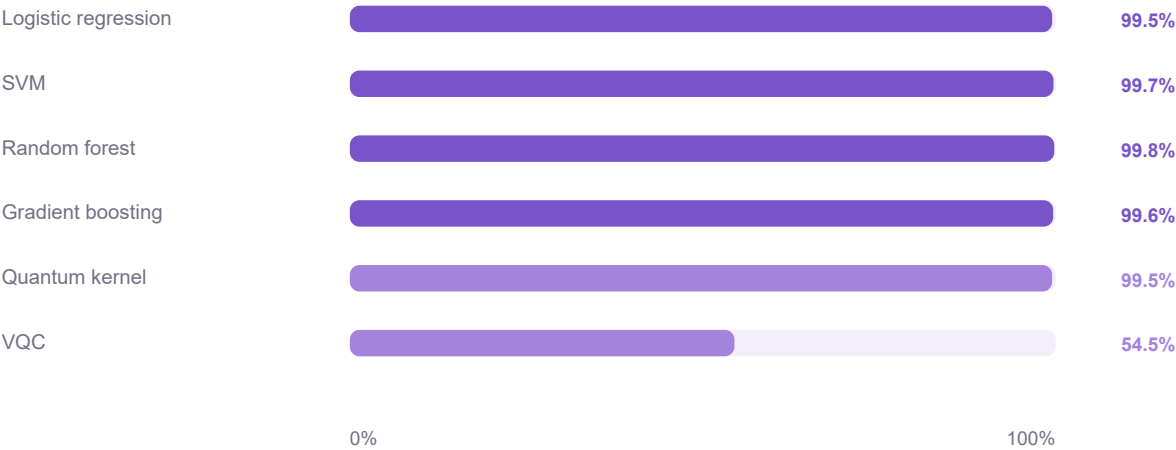
15 / PERSISTED EXPERIMENT EVIDENCE

Measured benchmark results

Experiment: 00f2897d0d744df59298d38e83f3525e
Created: 2026-10-01T04:04:51Z (UTC)
Source: local SQLite records; latest complete six-model run.

MODEL	ACC %	SENS %	SPEC %	F1	AUC	AP
Logistic	97.20	94.34	98.89	0.9615	0.9950	0.9927
SVM	99.30	98.11	100.00	0.9905	0.9969	0.9958
Forest	97.20	92.45	100.00	0.9608	0.9979	0.9966
Boosting	97.20	92.45	100.00	0.9608	0.9964	0.9953
Q. kernel	93.01	83.02	98.89	0.8980	0.9950	0.9913
VQC	65.03	24.53	88.89	0.3421	0.5449	0.5070

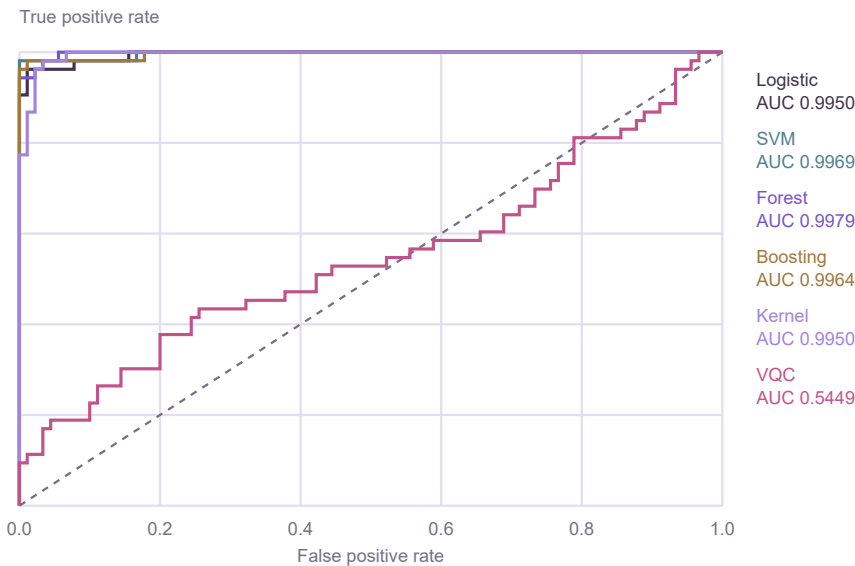
ROC-AUC by model



MODEL	TRAIN ROWS	TEST ROWS	TRAIN (s)	INFER (ms/row)
Logistic	426	143	0.019	0.023
SVM	426	143	0.030	0.043
Forest	426	143	0.302	0.083
Boosting	426	143	1.997	0.046
Q. kernel	160	143	0.020	0.063
VQC	160	143	0.104	0.077

AP = average precision. Train and inference values are wall-clock measurements from this run. Two-qubit VQC results with a ten-evaluation budget are smoke-test evidence rather than an optimized research result.

Results interpretation and explainability



MODEL	TN	FP	FN	TP	PREC %
Logistic	89	1	3	50	98.04
SVM	90	0	1	52	100.00
Forest	90	0	4	49	100.00
Boosting	90	0	4	49	100.00
Q. kernel	89	1	9	44	97.78
VQC	80	10	40	13	56.52

SVM makes one error on this split: one malignant sample is predicted negative. Random forest has the largest AUC but misses four positives at the fixed threshold. The kernel ranks examples strongly (AUC 0.9950) while missing nine positives. VQC misses 40 of 53 positives and has AUC near the chance-ranking level. Its poor result must remain visible rather than being hidden by an aggregate quantum label.

Feature-importance method

Permutation importance is computed on the first up to 40 held-out rows, with two shuffles per feature and accuracy as the scoring function. A positive value means shuffling reduced accuracy; negative values can occur through sampling variability. Correlated features can share information, making individual scores small even when the feature group matters. The interface clips negative bar widths to zero while preserving the numeric value.

Prediction responses attach the model's five largest global importance values. These values were not calculated specifically for the submitted sample. SHAP, LIME and causal attribution remain future work. Using held-out rows to inspect importance is appropriate for exploratory diagnosis, but iteratively changing the model from those findings also consumes the independence of the holdout.

Prediction workflow

From a saved model to a new sample

The user selects a model and submits a JSON array of one to 100 feature dictionaries. The request must contain exactly the feature columns stored for that model's dataset. The backend loads the saved joblib pipeline, constructs a DataFrame and calls predict_proba. The returned class is determined by the same 0.5 threshold used in evaluation.

```
POST /api/models/predict
{
  "model_id": "<saved model identifier>",
  "samples": [{"<feature 1>": 12.5, "<feature 2>": 0.3}]
}
Use all actual feature columns, not this abbreviated example.
```

RESPONSE FIELD	MEANING
positive_class	The label corresponding to probability column 1.
predictions[].class	Negative or positive label at the fixed 0.5 threshold.
predictions[].probability	Estimated positive-class probability from this model.
explanation	Top five global permutation-importance records.
note	Explicitly identifies probability as a model estimate, not clinical confidence.

Demonstration and validation

The interface can load a public benchmark row with its target removed. Browser verification exercised loading that row and displaying a returned prediction. The API integration test submits a benchmark sample to all six fitted model types and verifies successful inference and valid probability ranges.

Practical limitations

Benchmark preview samples may be training rows, so predicting one does not demonstrate generalization. The union-of-columns check on a batch does not guarantee each row individually supplied every feature; absent entries can become missing values and be imputed. Inference errors are returned as a generic invalid-values message. Batch CSV inference and row-specific explanation charts are not yet implemented.

Probability versus confidence

A percentage on screen is an estimator output for the positive class. It is not a calibrated clinical risk estimate, confidence interval, diagnosis or assurance that the result is correct.

Testing and quality assurance

CHECK	EVIDENCE / RESULT
Production frontend build	npm run build completed successfully after final frontend edits; TypeScript and Vite build passed.
Python integration / unit suite	2 tests passed; final recorded run took 12.16 seconds.
All-model integration flow	Trains all six models; waits for completion; validates metrics, confusion counts and saved-model inference.
Quantum unit checks	State normalization, probabilities in [0,1], sums equal one and seeded VQC weight reproducibility.
API negative cases	Invalid upload extension and invalid sample columns return 422.
Legacy JavaScript tests	28 existing campus tests passed; these do not cover the new research frontend.
Browser verification	Checked rendered overview, loaded benchmark JSON and displayed a real prediction.
Container deployment	Dockerfile supplied; image build/run was not verified.

What the tests prove

The tests demonstrate executable integration among API, dataset storage, classical/quantum fitting, metric persistence and inference. They assert internal consistency and bounded probabilities. They do not certify statistical validity, anonymization, clinical performance or complete production security.

Coverage still needed

- Dedicated frontend component and browser tests for upload, configuration, experiment status, results and accessibility.
- Temporary isolated storage for tests: the current backend test writes experiments into the local application database.
- More cases for malformed CSV, duplicate headers, non-finite features, rare labels after deduplication, batch-row completeness and high-cardinality categories.
- Concurrency, database locking, process restart, interrupted artifact writes and multi-worker behavior.
- Numerical equivalence tests against an independent quantum framework, noise tests and hardware validation if those integrations are added.

Security, privacy and deployment

Implemented safeguards

Upload size and extension checks, CSV parsing checks, Pydantic numeric bounds, server-generated storage identifiers, parameterized SQL and generic client-facing failure messages provide a useful local baseline. Model and dataset lookups require known records. The default launch command binds to 127.0.0.1 rather than exposing the service to the network.

Controls that are absent

There is no login, authorization, per-user tenant separation, dataset anonymization scanner, encryption-at-rest policy, retention workflow, rate limiting or clinical audit trail. The displayed profile is not an identity boundary. All connected local clients can access the same records. A .csv suffix does not independently establish safe or suitable content.

Joblib loading must remain restricted to locally generated trusted artifacts because Python serialization can execute code when loading untrusted files. Server logs do not intentionally print sample values, but exception logging can contain data-related details; a production design would need explicit redaction rules. Dataset previews and local CSV files retain uploaded content.

Deployment modes

MODE	OPERATING MODEL
Development	Vite :3000 proxies /api to FastAPI :5000; start with npm run dev.
Built local application	Build dist, then run FastAPI; the backend serves static frontend files on :5000.
Docker scaffold	Multi-stage Node build + Python runtime; map port 5000 and persist /app/backend/storage. Not executed in verification.

Before shared or public deployment

Add authentication and authorization, HTTPS, explicit origin policy, encrypted storage, upload limits and resource isolation, retention/deletion controls, atomic job ownership, monitored backups and an appropriate privacy review. The current server should remain in a trusted local research environment.

20 / TECHNICAL PROJECT REPORT

Installation and operating guide

Prerequisites and first run

Use Python 3.10 or newer and Node.js 20 or newer. Run commands from the project root. A Python virtual environment is recommended to isolate dependencies; the project uses requirements ranges rather than exact locks.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r backend/requirements.txt
npm install
npm run dev
```

Development UI: <http://localhost:3000>

API documentation: <http://localhost:5000/docs>

Readiness endpoint: <http://localhost:5000/api/health>

Repeat the verified workflow

- Open Datasets and select Wisconsin Breast Cancer, or specify a target name before uploading an anonymized CSV.
- Set seed and split; choose PCA or ANOVA and bounded quantum settings. Select the models you want to compare.
- Start training and follow Experiments. If a run fails, inspect server logs without sharing patient data.
- Review metrics and positive-class semantics in Evaluation. Choose a model for ROC and confusion details.
- Inspect global importance and run a benchmark or new JSON sample in Predictions. Export JSON results.

Build, test and recover

```
npm run build
npm test
python -m pytest backend/test_platform.py -q
npm run dev:backend
```

If the UI reports backend offline, check port 5000 and installed Python packages. If a restarted run is marked failed, start a new experiment; there is no resume operation. Back up the SQLite database, corresponding CSV files and corresponding joblib files together. Do not delete one artifact type while retaining inconsistent metadata.

Limitations, roadmap and conclusion

Scientific and engineering limitations

The current system evaluates one public dataset with one split. It does not assess prospective detection, population transportability, patient-level grouping or multi-site shift. Quantum/classical training budgets differ, and the current kernel's fixed final unitary leaves exact overlaps unchanged. VQC is evaluated with a very small optimization budget. There is no shot noise, device noise or hardware execution.

Explainability is a lightweight global permutation diagnostic. There are no calibration studies, confidence intervals, cross-validation, memory profiling or controlled equal-budget baselines. UI dataset quality counts do not replace domain review. The single-process worker and local storage are sufficient for demonstration but do not provide production-grade multi-user service guarantees.

PRIORIT Y	NEXT DEVELOPMENT STEP	EXPECTED BENEFIT
1	Equal-budget, repeated-split evaluation; untouched final set	More defensible model comparisons.
2	Input-dependent quantum maps and independent simulator checks	A meaningful investigation of quantum feature structure.
3	Calibration, confidence intervals and threshold validation	More transparent uncertainty and trade-offs.
4	Authentication, tenant isolation and durable job queue	Safe shared-workspace operation.
5	SHAP/LIME, row-specific explanations and test isolation	Clearer diagnostics and stronger verification.
6	Additional disease benchmarks and external validation	Broader task coverage and generalization evidence.
7	Hardware adapters, noise models and locked environments	Reproducible quantum execution research.

Conclusion

Qura delivers the core engineering loop: load data, fit reproducible pipelines, execute classical and simulated quantum models, compare measured results, preserve artifacts and run new-sample inference. The strongest current evidence supports the functionality of that loop and strong classical baselines on this benchmark. The honest next step is a more rigorous and equitable study, not a claim of clinical readiness or quantum advantage.

Appendix: configuration, source map and references

Exact reported experiment configuration

```
{
  "dataset_id": "wisconsin",
  "models": [
    "Logistic regression",
    "Support vector machine",
    "Random forest",
    "Gradient boosting",
    "Quantum kernel",
    "Variational quantum classifier"
  ],
  "seed": 42,
  "test_size": 0.25,
  "qubits": 2,
  "depth": 1,
  "iterations": 10,
  "balance": true,
  "selection": "pca"
}
```

Source map

PATH	CONTENTS
src/App.tsx	Loads the new research app.
src/ResearchApp.tsx / research.css	Research UI logic, navigation, forms and responsive styling.
backend/main.py	FastAPI routes, persistence, preprocessing, training and metrics.
backend/quantum.py	Exact statevector, kernel classifier and VQC.
backend/test_platform.py	Quantum and all-model integration checks.
backend/storage/	Local database, datasets and fitted pipelines (Git-ignored).
Dockerfile / README.md	Container scaffold and setup/methodology documentation.

The two-qubit, one-layer, ten-evaluation experiment differs from the default UI configuration (four qubits, two layers, thirty evaluations). The report does not substitute one configuration for the other.

Appendix: references and glossary

Project evidence

- [P1] User-supplied master project development brief: Hybrid Quantum-Classical Machine Learning Platform for Early Disease Detection. Requirements source; not evidence of implemented functionality.
- [P2] Workspace source files listed in the preceding appendix, inspected for this report on 1 October 2026.
- [P3] SQLite persisted experiment 00f2897d0d744df59298d38e83f3525e, created 2026-10-01T04:04:51Z; model artifacts and metrics. The six-model run is the source of every numerical performance result in this report.
- [P4] Verification outputs from the development session: successful TypeScript/Vite production build; Python tests 2 passed; legacy Vitest tests 28 passed; manual browser sample-inference check.

Official technical references

- [R1] scikit-learn: load_breast_cancer
https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html
- [R2] scikit-learn: Common pitfalls and recommended practices
https://scikit-learn.org/stable/common_pitfalls.html
- [R3] SciPy: minimize, method COBYLA
<https://docs.scipy.org/doc/scipy/reference/optimize.minimize-cobyla.html>

Official documentation was consulted for benchmark dimensions, the training-only transformation principle and the meaning of COBYLA maxiter. Source-code behavior in the inspected workspace takes precedence over newer library documentation.

Glossary

TERM	MEANING
Hybrid model	A workflow combining classical transforms/optimization with quantum-state features or circuits.
Qubit / statevector	A quantum wire; a vector of complex amplitudes describing a pure simulated state.
PCA / ANOVA	Unsupervised principal components / supervised statistical feature selection.
VQC / kernel	A trainable variational circuit / a pairwise similarity function for an SVM.
ROC-AUC / AP	Ranking discrimination measure / average precision over the recall curve.
Holdout / leakage	Separated test rows / information entering training that should have been unavailable.
Checkpoint	A serialized fitted pipeline used for later inference.